Unidad 5
Persistencia: DAO y Singleton

Prof: Lic. Rosales Pablo (prosales@unpata.edu.ar)
Materia: Programación Orientada a Objetos
Año: 2026

Persistir es guardar afuera del proceso y reconstruir los objetos después.

El problema de diseño es que el modelo no se entere de cómo.

SQLite

- Toda la base en un único archivo. Sin servidor ni instalación.
- Desde Java, con JDBC, la misma interfaz que para cualquier otra base.
- El driver es **`sqlite-jdbc-3.50.3.0.jar`**, que va en la carpeta **`lib`** del proyecto.
- Con Java 22 o más nuevo aparecen avisos de acceso nativo al cargarlo. Son avisos, no errores.

"No suitable driver found": el jar no está en el classpath.

JDBC en cuatro clases

```
Connection conexion = DriverManager.getConnection("jdbc:sqlite:juego.db");

String sql = "SELECT jugador, puntos FROM puntajes ORDER BY puntos DESC
LIMIT ?";
try (PreparedStatement sentencia = conexion.prepareStatement(sql)) {
    sentencia.setInt(1, 10);                          // el hueco
    try (ResultSet filas = sentencia.executeQuery()) {
        while (filas.next()) {
            String jugador = filas.getString("jugador");
            int puntos = filas.getInt("puntos");
        }
    }
} catch (SQLException e) { ... }
```

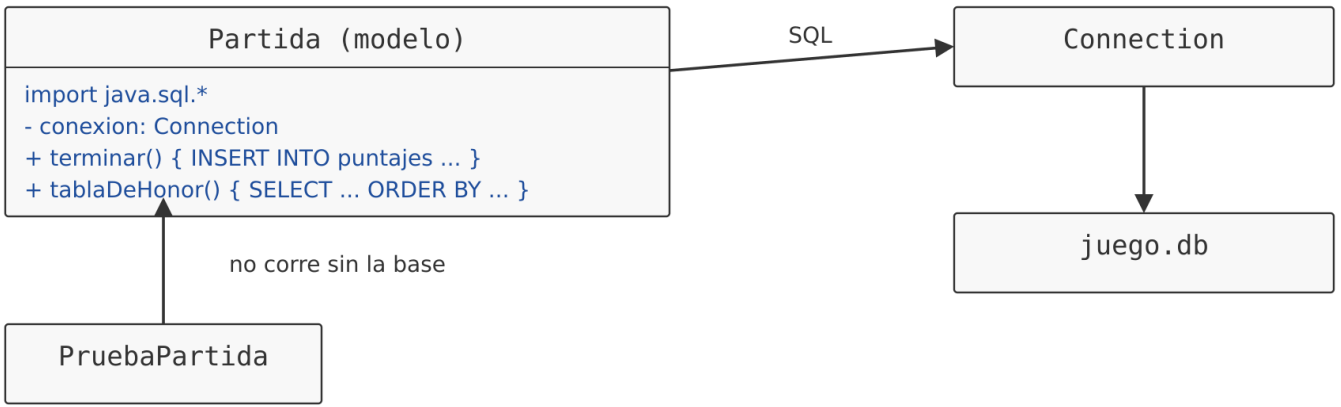
`Connection`, `PreparedStatement`, `ResultSet` y `SQLException`. Los tres primeros se cierran con un try con recursos.

Los datos van en los huecos

- Pegado con **+**: **`"INSERT INTO puntajes VALUES ('" + jugador + "', ...)"`**.
- Un jugador que se llame **`Ana'); DROP TABLE puntajes; --`** borra la tabla.
- Con **`setString()`**, el valor viaja aparte de la sentencia y la base lo trata como un dato, sea lo que sea.

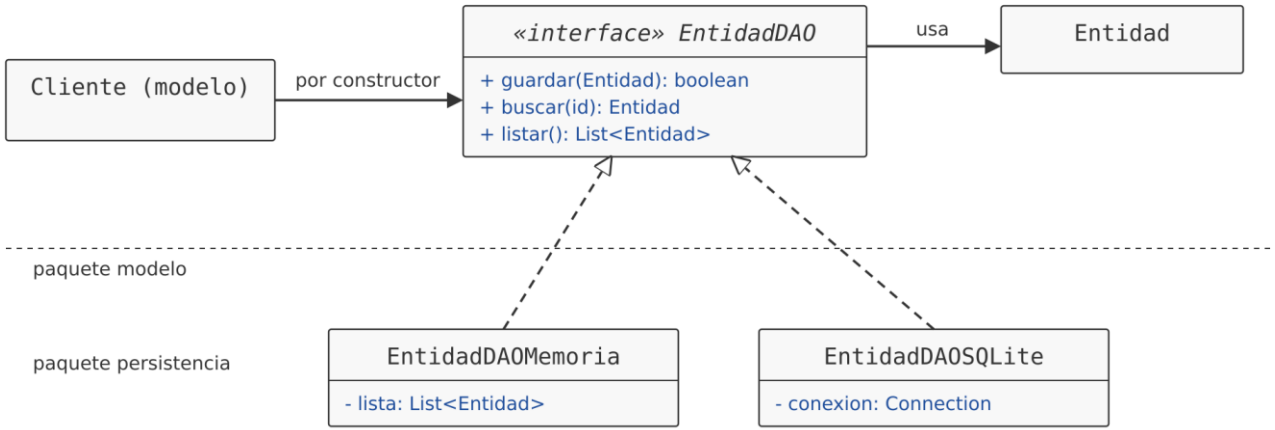
Nunca armar SQL concatenando datos.

DAO: el problema



El modelo importa `java.sql`, conoce las tablas, y no se puede probar sin la base.

DAO: la solución



La interfaz vive en el modelo. La implementación con JDBC vive aparte, y el cliente no sabe cuál tiene.

La interfaz, en términos del dominio

```
package modelo;

public interface PuntajeDAO {
    boolean guardar(Puntaje puntaje);
    List<Puntaje> mejores(int cantidad);
}

public class Partida {
    private final PuntajeDAO puntajes;          // por constructor

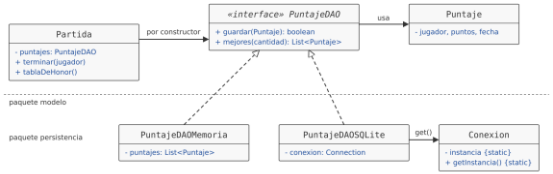
    public boolean terminar(String jugador) {
        return puntajes.guardar(new Puntaje(jugador, puntos, LocalDate.now()));
    }
}
```

Ni una palabra de SQL. Es inversión de dependencias: el modelo define la interfaz y la base se adapta.

La implementación, aparte

```
public class PuntajeDAOSQLite implements PuntajeDAO {
  private final Connection conexion;   // por
constructor

  public boolean guardar(Puntaje p) {
    String sql = "INSERT INTO puntajes "
        + "(jugador, puntos, fecha) VALUES (?, ?, ?)";
    try (PreparedStatement s =
conexion.prepareStatement(sql)) {
      s.setString(1, p.getJugador());
      s.setInt(2, p.getPuntos());
      s.setString(3, p.getFecha().toString());
      return s.executeUpdate() == 1;
    } catch (SQLException e) { return false; }
  }
}
```



La única clase que sabe que hay una tabla. La versión en memoria no importa `java.sql`, y sirve para probar.

Singleton para la conexión, y nada más

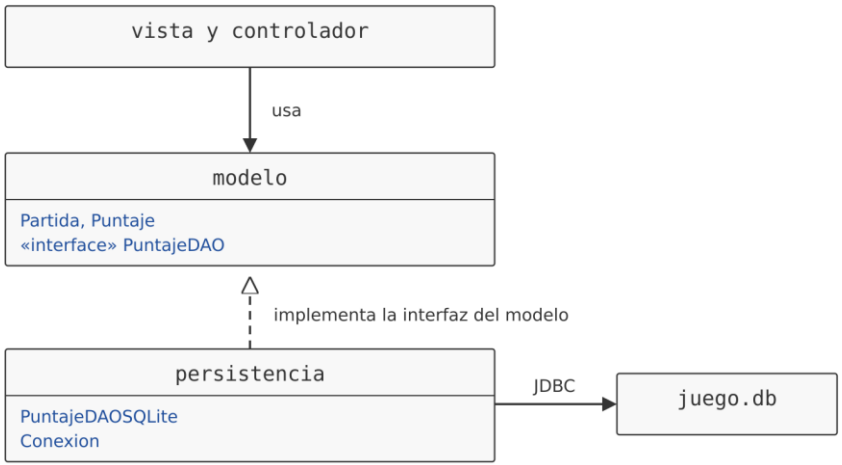
```
public class Conexion {
    private static Conexion instancia;
    private final Connection conexion;

    private Conexion() throws SQLException {
        conexion = DriverManager.getConnection("jdbc:sqlite:juego.db");
    }
    public static Conexion getInstancia() throws SQLException { ... }
    public Connection get() { return conexion; }
}

// en main, una sola vez
PuntajeDAO puntajes = new PuntajeDAOSQLite(Conexion.getInstancia().get());
```

Dos conexiones al mismo archivo SQLite se bloquean. Ningún DAO llama a `getInstancia()`: la reciben.

Las capas



La persistencia depende del modelo, porque implementa una interfaz que el modelo definió. El modelo no depende de nadie.

Ventajas y desventajas

- Cambiar de motor, o pasar de un archivo a una base, es escribir otra implementación.
- El modelo se prueba sin base, y el SQL de cada tabla está en una sola clase.
- Es una capa más: una interfaz y una o dos clases por entidad.
- La interfaz tiende a llenarse de consultas particulares, y cada una va en todas las implementaciones.

Con más de dos tablas, el paso siguiente es un DAO genérico, `DAO<T>`.

Para el proyecto

- Definir qué se guarda: una o dos entidades, con sus atributos.
- Escribir la interfaz DAO de cada una en el paquete del modelo.
- Implementarla primero en memoria, para que el juego funcione sin base, y después con SQLite.
- Obtener la conexión con el Singleton en `main` y pasarla a los DAO por constructor.

Si alguna clase del paquete `modelo` importa `java.sql`, algo quedó en el lugar equivocado.

La actividad de hoy

- Una partida que guarda puntajes con el SQL adentro y abre una conexión propia, y una prueba que solo corre contra la base real.
- **Parte 1**: ejecutar, contar las conexiones, y ver a "Prueba" en el primer lugar de la tabla de honor del juego.
- **Parte 2**: `Conexion` Singleton, `PuntajeDAO`, la versión SQLite y la de memoria.
- **Parte 3**: una consulta nueva, `mejoresDe()`, y contar cuántos archivos toca.